

BankManagement

Selected SQL Queries from Stored Procedures and Triggers

Categorized Technical Query Reference

Source scope	Current BankManagement stored procedures and triggers
Included material	SELECT, INSERT, UPDATE, DELETE, MERGE, EXISTS, and state-transition blocks reproduced from project source
Excluded material	Standalone test commands, diagnostic queries, API examples, schema-installation commands, and newly composed SQL
Organization	Authentication, customers, accounts, transactions, loans, employees, and integrity triggers

All SQL excerpts are copied from the current project source files. Source paths and line ranges are provided for verification.

1. Scope and organization

This document presents the most significant SQL operations implemented inside the BankManagement stored procedures and triggers. The excerpts are grouped by functional responsibility. Each item identifies the database object, the source file, the original line range, and the role of the query within the application.

The excerpts preserve the SQL written in the project. Ellipses and substitute statements are not used. Where an operation spans more than one important block, the relevant blocks are listed separately under the same functional category.

2. Category index

Category	Number of excerpts	Primary subjects
Authentication and session management	6	Login, effective roles, session creation, validation, branch context, logout
Customer operations	3	Scoped search, contact updates, soft deactivation
Account operations	7	Portfolio query, opening, lifecycle states, dormancy, interest
Financial transactions	8	Pending creation, available balance, finalization, reversal
Loans and installments	5	Loan schedule, status reporting, payment link, daily state maintenance
Employees, managers, and transfers	5	Suspension, reactivation, manager role restoration, branch transfer
Integrity triggers	6	Pending-operation protection, ledger synchronization, state synchronization, interval and type rules

3. Authentication and session management

3.1 Resolve the application user from the submitted password and identifier

Database object	dbo.sp_User_Login
Source file	StoredProcedures/UserProcedures/ Procedure_User_Login.sql
Source lines	137-153

Function. Hashes the submitted password and resolves UserID, CustomerID, and EmployeeID from an active Users row. The supported identifier fields are evaluated inside the fixed WHERE clause.

```

DECLARE @PasswordHash CHAR(64);
SET @PasswordHash = dbo.fn_HashPassword(@Password);

SELECT
    @UserID = U.UserID,
    @CustomerID = U.CustomerID,
    @EmployeeID = U.EmployeeID
FROM dbo.Users AS U
WHERE U.PasswordHash = @PasswordHash
AND U.IsActive = 1
AND
(
    (@CleanUsername IS NOT NULL AND U.Username = @CleanUsername)
OR (@LoginUserID IS NOT NULL AND U.UserID = @LoginUserID)
OR (@LoginCustomerID IS NOT NULL AND U.CustomerID = @LoginCustomerID)
OR (@LoginEmployeeID IS NOT NULL AND U.EmployeeID = @LoginEmployeeID)
);

```

3.2 Calculate effective Employee and Admin roles

Database object	dbo.sp_User_Login
Source file	StoredProcedures/UserProcedures/ Procedure_User_Login.sql
Source lines	246-309

Function. Distinguishes assigned roles from currently effective roles by checking employment status, administrative capability, and manager job title before building the role list returned at login.

```

DECLARE @EmployeeIsActive BIT,
        @AdminIsEffective BIT;

SET @EmployeeIsActive = 0;
SET @AdminIsEffective = 0;

IF @HasEmployee = 1
AND @EmployeeID IS NOT NULL
AND EXISTS
(
    SELECT 1
    FROM dbo.Employee AS E
    WHERE E.EmployeeID = @EmployeeID
    AND E.EmpStatus = N'Active'
)
BEGIN
    SET @EmployeeIsActive = 1;
END;

```

```

IF @HasAdmin = 1
  AND @EmployeeIsActive = 1
  AND EXISTS
  (
    SELECT 1
    FROM dbo.Employee AS E
    WHERE E.EmployeeID = @EmployeeID
      AND E.EmpStatus = N'Active'
      AND E.CanAccessAdmin = 1
      AND E.JobTitle IN (N'Branch Manager', N'Vice Manager')
  )
BEGIN
  SET @AdminIsEffective = 1;
END;

-----
-- 7. Build effective role list returned to the application.
-----

DECLARE @RoleList TABLE
(
  RoleName NVARCHAR(50) NOT NULL PRIMARY KEY
);

INSERT INTO @RoleList(RoleName)
VALUES(N'Customer');

IF @EmployeeIsActive = 1
  INSERT INTO @RoleList(RoleName) VALUES(N'Employee');

IF @AdminIsEffective = 1
  INSERT INTO @RoleList(RoleName) VALUES(N'Admin');

IF @HasHighAdmin = 1
  INSERT INTO @RoleList(RoleName) VALUES(N'HighAdmin');

SELECT @Roles = STUFF
(
  (
    SELECT N',' + RoleName
    FROM @RoleList
    ORDER BY RoleName
    FOR XML PATH(""), TYPE
  ).value('.', 'NVARCHAR(MAX)'),
  1, 1, N''
);

```

3.3 Close expired sessions and create a new fixed-lifetime session

Database object	dbo.sp_User_Login
Source file	StoredProcedures/UserProcedures/ Procedure_User_Login.sql
Source lines	315-348

Function. Marks previously expired sessions inactive, creates an opaque token, and inserts a new Sessions row with an absolute ExpiresAt value.

```

DECLARE @LoginTime DATETIME;
SET @LoginTime = GETDATE();
SET @ExpiresAt = DATEADD(MINUTE, @SessionTtlMinutes, @LoginTime);

-- Opportunistically close previously expired sessions so the
-- IsActive flag remains consistent even if they are never used again.
UPDATE dbo.Sessions
SET IsActive = 0,
    LogoutTime = COALESCE(LogoutTime, @LoginTime)
WHERE IsActive = 1
  AND ExpiresAt <= @LoginTime;

SET @SessionToken = CONVERT(NVARCHAR(36), NEWID())

```

```

+ N'-'
+ CONVERT(NVARCHAR(36), NEWID());

INSERT INTO dbo.Sessions
(
    UserID,
    SessionToken,
    LoginTime,
    ExpiresAt,
    LogoutTime,
    IsActive
)
VALUES
(
    @UserID,
    @SessionToken,
    @LoginTime,
    @ExpiresAt,
    NULL,
    1
);

```

3.4 Validate a bearer session and record expiration

Database object	dbo.sp_User_ValidateSession
Source file	StoredProcedures/Session/ Procedure_User_ValidateSession.sql
Source lines	109-167

Function. Resolves the token, checks active and expiration state, deactivates an expired session, and records the expiration in AuditLog.

```

SET @Now = GETDATE();

SELECT
    @SessionID = S.SessionID,
    @UserID = U.UserID,
    @CustomerID = U.CustomerID,
    @EmployeeID = U.EmployeeID,
    @LoginTime = S.LoginTime,
    @ExpiresAt = S.ExpiresAt,
    @SessionIsActive = S.IsActive,
    @UserIsActive = U.IsActive,
    @Username = U.Username
FROM dbo.Sessions AS S
INNER JOIN dbo.Users AS U
    ON U.UserID = S.UserID
WHERE S.SessionToken = @CleanSessionToken;

IF @SessionID IS NULL
BEGIN
    RAISERROR('Invalid session token.', 16, 1);
    RETURN;
END;

IF @SessionIsActive = 0
BEGIN
    RAISERROR('Session is inactive. Please sign in again.', 16, 1);
    RETURN;
END;

IF @ExpiresAt IS NULL OR @ExpiresAt <= @Now
BEGIN
    UPDATE dbo.Sessions
    SET IsActive = 0,
        LogoutTime = COALESCE(LogoutTime, @Now)
    WHERE SessionID = @SessionID
    AND IsActive = 1;

```

```

INSERT INTO dbo.AuditLog
(
    UserID,
    ActionType,
    TableName,
    RecordID,
    ActionDate,
    Details
)
VALUES
(
    @UserID,
    N'SessionExpired',
    N'Sessions',
    @SessionID,
    @Now,
    N'Session expired after its fixed lifetime.'
);

RAISERROR('Session expired. Please sign in again.', 16, 1);
RETURN;
END;

```

3.5 Resolve current branch and reconstruct effective roles

Database object	dbo.sp_User_ValidateSession
Source file	StoredProcedures/Session/ Procedure_User_ValidateSession.sql
Source lines	297-385

Function. Reads the current Employee and EMPB state, resolves the active branch, rebuilds the effective role set, and optionally enforces a required role.

```

IF @EmployeeID IS NOT NULL
BEGIN
    SELECT
        @EmployeeStatus = E.EmpStatus,
        @JobTitle = E.JobTitle,
        @CanAccessAdmin = E.CanAccessAdmin
    FROM dbo.Employee AS E
    WHERE E.EmployeeID = @EmployeeID;

    SELECT TOP (1)
        @CurrentBranchID = EB.BranchID,
        @CurrentBranchName = B.BranchName
    FROM dbo.EMPB AS EB
    INNER JOIN dbo.Branch AS B
        ON B.BranchID = EB.BranchID
    WHERE EB.EmployeeID = @EmployeeID
        AND EB.WorkingStatus = N'Working'
        AND EB.EndDate IS NULL
    ORDER BY EB.StartDate DESC, EB.EMPBID DESC;
END;

IF @HasEmployee = 1
    AND @EmployeeID IS NOT NULL
    AND @EmployeeStatus = N'Active'
BEGIN
    SET @EmployeeIsActive = 1;
END;

IF @HasAdmin = 1
    AND @EmployeeIsActive = 1
    AND @CanAccessAdmin = 1
    AND @JobTitle IN (N'Branch Manager', N'Vice Manager')
BEGIN
    SET @AdminIsEffective = 1;
END;

```

```

IF @HasHighAdmin = 1
    AND @EmployeeID IS NULL
BEGIN
    SET @HighAdminIsEffective = 1;
END;

-----
-- 7. Build effective role list.
-----

DECLARE @RoleList TABLE
(
    RoleName NVARCHAR(50) NOT NULL PRIMARY KEY
);

INSERT INTO @RoleList(RoleName)
VALUES(N'Customer');

IF @EmployeeIsActive = 1
    INSERT INTO @RoleList(RoleName) VALUES(N'Employee');

IF @AdminIsEffective = 1
    INSERT INTO @RoleList(RoleName) VALUES(N'Admin');

IF @HighAdminIsEffective = 1
    INSERT INTO @RoleList(RoleName) VALUES(N'HighAdmin');

DECLARE @EffectiveRoles NVARCHAR(MAX);

SELECT @EffectiveRoles = STUFF
(
    (
        SELECT N',' + RoleName
        FROM @RoleList
        ORDER BY RoleName
        FOR XML PATH(""), TYPE
    ).value('.', 'NVARCHAR(MAX)'),
    1, 1, N''
);

-----
-- 8. Optional required-role authorization check.
-----

IF @CleanRequiredRole IS NOT NULL
    AND NOT EXISTS
    (
        SELECT 1
        FROM @RoleList
        WHERE RoleName = @CleanRequiredRole
    )
BEGIN
    RAISERROR('Session is valid, but the required role is not currently effective for this user.', 16, 1);
    RETURN;
END;

```

3.6 Deactivate a session during logout

Database object	dbo.sp_User_Logout
Source file	StoredProcedures/UserProcedures/ Procedure_User_Logout.sql
Source lines	48-92

Function. Locks the active session row, marks it inactive, sets LogoutTime, and writes a logout audit record.

```

SELECT
    @SessionID = S.SessionID,
    @UserID = S.UserID
FROM dbo.Sessions AS S WITH (UPDLOCK, HOLDLOCK)
WHERE S.SessionToken = @SessionToken

```

```

AND S.IsActive = 1;

IF @SessionID IS NULL
BEGIN
    RAISERROR('Invalid or inactive session token.', 16, 1);
    RETURN;
END;

SET @LogoutTime = GETDATE();

UPDATE dbo.Sessions
SET IsActive = 0,
    LogoutTime = @LogoutTime
WHERE SessionID = @SessionID
AND IsActive = 1;

INSERT INTO dbo.AuditLog
(
    UserID,
    ActionType,
    TableName,
    RecordID,
    ActionDate,
    Details
)
VALUES
(
    @UserID,
    N'UserLogout',
    N'Sessions',
    @SessionID,
    GETDATE(),
    N'User logged out.'
);

SELECT
    @UserID AS UserID,
    @SessionID AS SessionID,
    @LogoutTime AS LogoutTime,
    N'LoggedOut' AS Result;

```

4. Customer operations

4.1 Search customers with branch-aware visibility

Database object	dbo.sp_Customer_Search
Source file	StoredProcedures/CustomerProcedures/ Procedure_Customer_Search.sql
Source lines	136-164

Function. Applies optional customer filters and limits non-HighAdmin results to customers who own an account in the caller current branch.

```

SELECT
    C.CustomerID,
    C.FirstName,
    C.LastName,
    C.NationalID,
    C.BirthDate,
    C.Phone,
    C.Email,
    C.Address,
    C.RegistrationDate,
    C.IsActive
FROM dbo.Customer AS C
WHERE (@IncludeInactive = 1 OR C.IsActive = 1)

```



```

AND (@CleanNameSearch IS NULL OR C.FirstName LIKE @NamePattern OR C.LastName LIKE @NamePattern)
AND (@CleanNationalID IS NULL OR C.NationalID = @CleanNationalID)
AND (@CleanPhone IS NULL OR C.Phone = @CleanPhone)
AND (@CleanEmail IS NULL OR C.Email = @CleanEmail)
AND
(
  @IsHighAdminEffective = 1
  OR EXISTS
  (
    SELECT 1
    FROM dbo.Account AS A
    WHERE A.CustomerID = C.CustomerID
    AND A.BranchID = @CurrentBranchID
  )
)
)
ORDER BY C.RegistrationDate DESC, C.CustomerID DESC;

```

4.2 Update customer contact information after uniqueness checks

Database object	dbo.sp_Customer_Update
Source file	StoredProcedures/CustomerProcedures/ Procedure_Customer_Update.sql
Source lines	114-125

Function. Rejects duplicate phone or email values and updates only the permitted customer attributes.

```

IF EXISTS (SELECT 1 FROM dbo.Customer WHERE Phone = @Phone AND CustomerID <> @CustomerID)
BEGIN RAISERROR('Another customer already uses this phone number.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;
IF EXISTS (SELECT 1 FROM dbo.Customer WHERE Email = @Email AND CustomerID <> @CustomerID)
BEGIN RAISERROR('Another customer already uses this email.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;

UPDATE dbo.Customer
SET FirstName = LTRIM(RTRIM(@FirstName)),
    LastName = LTRIM(RTRIM(@LastName)),
    Phone = LTRIM(RTRIM(@Phone)),
    Email = LTRIM(RTRIM(@Email)),
    Address = @Address
WHERE CustomerID = @CustomerID;

```

4.3 Soft-deactivate a customer, linked users, and active sessions

Database object	dbo.sp_Customer_Delete
Source file	StoredProcedures/CustomerProcedures/ Procedure_Customer_Delete.sql
Source lines	109-126

Function. Changes Customer.IsActive, disables linked application users, and terminates their active sessions without deleting historical banking records.

```

UPDATE dbo.Customer
SET IsActive = 0
WHERE CustomerID = @CustomerID;

DECLARE @DeactivatedUsers TABLE(UserID INT PRIMARY KEY);

UPDATE dbo.Users
SET IsActive = 0
OUTPUT inserted.UserID INTO @DeactivatedUsers(UserID)
WHERE CustomerID = @CustomerID
AND IsActive = 1;

```

```

UPDATE S
SET IsActive = 0,
    LogoutTime = GETDATE()
FROM dbo.Sessions AS S
INNER JOIN @DeactivatedUsers AS DU ON DU.UserID = S.UserID
WHERE S.IsActive = 1;

```

5. Account operations

5.1 Return account portfolio data and available balance

Database object	dbo.sp_Account_Search
Source file	StoredProcedures/AccountProcedures/ Procedure_Account_Search.sql
Source lines	270-317

Function. Joins account, customer, branch, product, and pending-outgoing data. AvailableBalance is calculated as posted balance minus pending outgoing holds.

```

SELECT
    A.AccountID,
    A.AccountNumber,
    A.CustomerID,
    C.FirstName AS CustomerFirstName,
    C.LastName AS CustomerLastName,
    C.NationalID AS CustomerNationalID,
    C.Phone AS CustomerPhone,
    C.Email AS CustomerEmail,
    A.BranchID,
    B.BranchName,
    B.BranchCode,
    B.City AS BranchCity,
    A.AccountTypeID,
    AT.TypeName AS AccountTypeName,
    AT.MinBalance,
    AT.InterestRate,
    A.Balance,
    ISNULL(P.PendingOutgoingAmount, 0) AS PendingOutgoingAmount,
    A.Balance - ISNULL(P.PendingOutgoingAmount, 0) AS AvailableBalance,
    A.OpenDate,
    A.CloseDate,
    A.AccountStatus,
    A.FrozenPreviousStatus
FROM dbo.Account AS A
INNER JOIN dbo.Customer AS C
    ON C.CustomerID = A.CustomerID
INNER JOIN dbo.Branch AS B
    ON B.BranchID = A.BranchID
INNER JOIN dbo.AccountType AS AT
    ON AT.AccountTypeID = A.AccountTypeID
LEFT JOIN dbo.vw_AccountPendingOutgoing AS P
    ON P.AccountID = A.AccountID
WHERE (@AccountID IS NULL OR A.AccountID = @AccountID)
    AND (@CleanAccountNumber IS NULL OR A.AccountNumber = @CleanAccountNumber)
    AND (@CleanAccountNumberSearch IS NULL OR A.AccountNumber LIKE @AccountNumberPattern)
    AND (@CustomerID IS NULL OR A.CustomerID = @CustomerID)
    AND (@CleanCustomerNationalID IS NULL OR C.NationalID = @CleanCustomerNationalID)
    AND (@CleanCustomerNameSearch IS NULL OR C.FirstName LIKE @CustomerNamePattern OR C.LastName LIKE @CustomerNamePattern)
    AND (@BranchID IS NULL OR A.BranchID = @BranchID)
    AND (@CleanBranchCode IS NULL OR B.BranchCode = @CleanBranchCode)
    AND (@AccountTypeID IS NULL OR A.AccountTypeID = @AccountTypeID)
    AND (@CleanAccountTypeName IS NULL OR AT.TypeName = @CleanAccountTypeName)
    AND (@CleanAccountStatus IS NULL OR A.AccountStatus = @CleanAccountStatus)
    AND (@CleanAccountStatus IS NOT NULL OR @IncludeClosed = 1 OR A.AccountStatus <> N'Closed')
    AND (@MinBalance IS NULL OR A.Balance >= @MinBalance)

```

```
AND (@MaxBalance IS NULL OR A.Balance <= @MaxBalance)
ORDER BY A.AccountID;
```

5.2 Generate a 12-digit account number and insert the account

Database object	dbo.sp_Account_Open
Source file	StoredProcedures/AccountProcedures/ Procedure_Account_Open.sql
Source lines	159-192

Function. Uses the account-number sequence, formats the value to twelve digits, and inserts a new Active account at the product minimum balance.

```
DECLARE @SeqValue BIGINT;
SET @SeqValue = NEXT VALUE FOR dbo.seq_AccountNumber;

IF @SeqValue > 999999999999
BEGIN
    RAISERROR('Account number sequence exceeded the 12-digit limit.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

SET @AccountNumber = RIGHT(REPLICATE('0', 12) + CAST(@SeqValue AS NVARCHAR(12)), 12);

INSERT INTO dbo.Account
(
    AccountNumber,
    CustomerID,
    BranchID,
    AccountTypeID,
    Balance,
    AccountStatus,
    OpenDate
)
VALUES
(
    @AccountNumber,
    @CustomerID,
    @BranchID,
    @AccountTypeID,
    @MinBalance,
    'Active',
    CAST(GETDATE() AS DATE)
);

SET @AccountID = CONVERT(INT, SCOPE_IDENTITY());
```

5.3 Freeze an account while preserving its previous state

Database object	dbo.sp_Account_Freeze
Source file	StoredProcedures/AccountProcedures/ Procedure_Account_Freeze_Unfreeze.sql
Source lines	26-34

Function. Locks the account, stores the current Active or Dormant status in FrozenPreviousStatus, changes the account to Frozen, and records the action.

```
SELECT @CurrentStatus=AccountStatus FROM dbo.Account WITH (UPDLOCK,HOLDLOCK) WHERE AccountID=@AccountID;
IF @CurrentStatus IS NULL BEGIN RAISERROR('Account does not exist.',16,1); ROLLBACK TRANSACTION; RETURN; END;
IF @CurrentStatus='Closed' BEGIN RAISERROR('Closed account cannot be frozen.',16,1); ROLLBACK TRANSACTION; RETURN; END;
```

```

IF @CurrentStatus='Frozen' BEGIN RAISERROR('Account is already frozen.',16,1); ROLLBACK TRANSACTION; RETURN; END;
UPDATE dbo.Account SET AccountStatus='Frozen', FrozenPreviousStatus=@CurrentStatus WHERE AccountID=@AccountID;
INSERT INTO dbo.AuditLog(UserID,ActionType,TableName,RecordID,Details)
VALUES(@UserID,'AccountFrozen','Account',@AccountID,CONCAT('Previous status: ',@CurrentStatus,'. Reason:
',ISNULL(@ReasonDescription,'not provided')));
COMMIT TRANSACTION;
SELECT @AccountID AS AccountID, 'Frozen' AS AccountStatus, @CurrentStatus AS PreviousStatus;

```

5.4 Unfreeze an account and restore the previous state

Database object	dbo.sp_Account_Unfreeze
Source file	StoredProcedures/AccountProcedures/ Procedure_Account_Freeze_Unfreeze.sql
Source lines	55-63

Function. Restores Active or Dormant from FrozenPreviousStatus, clears the saved value, and records the unfreeze operation.

```

SELECT @CurrentStatus=AccountStatus, @RestoreStatus=ISNULL(FrozenPreviousStatus,'Active') FROM dbo.Account WITH
(UPDLOCK,HOLDLOCK) WHERE AccountID=@AccountID;
IF @CurrentStatus IS NULL BEGIN RAISERROR('Account does not exist.',16,1); ROLLBACK TRANSACTION; RETURN; END;
IF @CurrentStatus <> 'Frozen' BEGIN RAISERROR('Only a Frozen account can be unfrozen.',16,1); ROLLBACK TRANSACTION; RETURN; END;
IF @RestoreStatus NOT IN ('Active','Dormant') SET @RestoreStatus='Active';
UPDATE dbo.Account SET AccountStatus=@RestoreStatus, FrozenPreviousStatus=NULL WHERE AccountID=@AccountID;
INSERT INTO dbo.AuditLog(UserID,ActionType,TableName,RecordID,Details)
VALUES(@UserID,'AccountUnfrozen','Account',@AccountID,CONCAT('Restored status: ',@RestoreStatus,'. Reason:
',ISNULL(@ReasonDescription,'not provided')));
COMMIT TRANSACTION;
SELECT @AccountID AS AccountID, @RestoreStatus AS AccountStatus;

```

5.5 Close an account

Database object	dbo.sp_Account_Close
Source file	StoredProcedures/AccountProcedures/ Procedure_Account_Close.sql
Source lines	50-55

Function. Sets the final balance and lifecycle fields used by the current account-closing implementation.

```

UPDATE dbo.Account
SET Balance = 0,
    AccountStatus = 'Closed',
    CloseDate = CAST(GETDATE() AS DATE),
    FrozenPreviousStatus = NULL
WHERE AccountID = @AccountID;

```

5.6 Identify and mark dormant accounts

Database object	dbo.sp_Account_DormantSweep
Source file	StoredProcedures/AccountProcedures/ Procedure_Account_DormantSweep.sql
Source lines	33-76

Function. Finds Active accounts with no completed customer Deposit, Withdrawal, or Transfer during the configured period, excludes accounts with pending transactions, and changes qualifying accounts to Dormant.

```

SELECT
    a.AccountID,
    a.AccountNumber,
    a.OpenDate,
    lastact.LastCustomerActivityAt,
    DATEADD(MONTH, @MonthsInactive,
        CAST(ISNULL(lastact.LastCustomerActivityAt, a.OpenDate) AS DATE)) AS DormantDueDate
INTO #DormantCandidates
FROM dbo.Account a WITH (UPDLOCK, HOLDLOCK)
OUTER APPLY
(
    SELECT MAX(COALESCE(tr.CompletedAt, tr.TransactionDate)) AS LastCustomerActivityAt
    FROM dbo.Transactions tr
    INNER JOIN dbo.TransactionType tt ON tt.TransactionTypeID = tr.TransactionTypeID
    WHERE tr.TransactionStatus = 'Completed'
        AND tt.TypeName IN ('Deposit', 'Withdrawal', 'Transfer')
        AND (tr.FromAccountID = a.AccountID OR tr.ToAccountID = a.AccountID)
) lastact
WHERE a.AccountStatus = 'Active'
    AND DATEADD(MONTH, @MonthsInactive,
        CAST(ISNULL(lastact.LastCustomerActivityAt, a.OpenDate) AS DATE)) <= CAST(GETDATE() AS DATE)
    AND NOT EXISTS (
        SELECT 1
        FROM dbo.Transactions trp
        WHERE trp.TransactionStatus = 'Pending'
            AND (trp.FromAccountID = a.AccountID OR trp.ToAccountID = a.AccountID)
    );

UPDATE a
SET AccountStatus = 'Dormant',
    FrozenPreviousStatus = NULL
FROM dbo.Account a
INNER JOIN #DormantCandidates d ON d.AccountID = a.AccountID;

INSERT INTO dbo.AuditLog (UserID, ActionType, TableName, RecordID, Details)
SELECT
    NULL,
    'AccountMarkedDormant',
    'Account',
    d.AccountID,
    CONCAT('Account ', d.AccountNumber, ' marked Dormant after ', @MonthsInactive,
        ' months without completed customer-initiated transactions. Last activity: ',
        COALESCE(CONVERT(VARCHAR(30), d.LastCustomerActivityAt, 121), 'none'))
FROM #DormantCandidates d;

```

5.7 Apply monthly interest to eligible accounts

Database object	dbo.sp_Account_ApplyMonthlyInterest
Source file	StoredProcedures/AccountProcedures/ Procedure_Account_ApplyMonthlyInterest.sql
Source lines	65-123

Function. Builds an interest batch, inserts completed Interest transactions while capturing their identities, and applies each interest amount to Account.Balance.

```

SELECT
    a.AccountID,
    ROUND(a.Balance * at.InterestRate / 100.0, 2) AS InterestAmount
INTO #InterestBatch
FROM dbo.Account a
INNER JOIN dbo.AccountType at ON a.AccountTypeID = at.AccountTypeID
WHERE a.AccountStatus = 'Active'
    AND a.Balance > 0

```

```

AND at.InterestRate > 0;

DELETE FROM #InterestBatch WHERE InterestAmount <= 0;

IF NOT EXISTS (SELECT 1 FROM #InterestBatch)
BEGIN
    COMMIT TRANSACTION;

    SELECT
        CAST(NULL AS INT)      AS AccountID,
        CAST(NULL AS DECIMAL(18,2)) AS InterestAmount,
        CAST(NULL AS INT)      AS TransactionID
    WHERE 1 = 0;

    RETURN;
END;

-----
-- Step 2: insert one already-Completed Transactions row
-- per account, capturing the generated TransactionID.
-----

IF OBJECT_ID('tempdb..#InterestTx') IS NOT NULL DROP TABLE #InterestTx;
CREATE TABLE #InterestTx (AccountID INT, TransactionID INT, InterestAmount DECIMAL(18,2));

DECLARE @Now DATETIME = GETDATE();

-- MERGE used purely to capture, per source row, the
-- IDENTITY value generated by each corresponding insert
-- (OUTPUT on a plain multi-row INSERT cannot be joined
-- back to the source rows by anything other than insertion
-- order, which is not guaranteed).
MERGE INTO dbo.Transactions AS tgt
USING #InterestBatch AS src
ON 1 = 0 -- never match; this MERGE is insert-only
WHEN NOT MATCHED THEN
    INSERT (TransactionTypeID, FromAccountID, ToAccountID, EmployeeID, Amount,
        TransactionDate, TransactionStatus, ReadyToCompleteAt, CompletedAt, Description)
    VALUES (@InterestTypeID, NULL, src.AccountID, NULL, src.InterestAmount,
        @Now, 'Completed', @Now, @Now, N'Monthly interest credit')
OUTPUT src.AccountID, inserted.TransactionID, src.InterestAmount
INTO #InterestTx (AccountID, TransactionID, InterestAmount);

-----
-- Step 3: apply the balance effect. This UPDATE is what
-- fires TR_Account_SyncBranchLedger, which propagates
-- the change into Branch.Balance / BranchLedger.
-----

UPDATE a
SET a.Balance = a.Balance + it.InterestAmount
FROM dbo.Account a
INNER JOIN #InterestTx it ON a.AccountID = it.AccountID;

```

6. Financial transactions

6.1 Queue a deposit transaction

Database object	dbo.sp_Transaction_Deposit
Source file	StoredProcedures/TransactionProcedures/ Desposit_SP.sql
Source lines	67-87

Function. Validates the destination account state, calculates ReadyToCompleteAt from the amount, and inserts a Pending Deposit transaction.

```

SELECT @AccountStatus = AccountStatus
FROM dbo.Account WITH (UPDLOCK, HOLDLOCK)

```

```

WHERE AccountID = @AccountID;

IF @AccountStatus NOT IN ('Active', 'Dormant')
BEGIN RAISERROR('Deposits are not allowed on a Closed or Frozen account.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;

DECLARE @TransactionTypeID INT;
SELECT @TransactionTypeID = TransactionTypeID FROM dbo.TransactionType WHERE TypeName = 'Deposit';
IF @TransactionTypeID IS NULL
BEGIN RAISERROR('Deposit transaction type is not configured.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;

DECLARE @DelaySeconds INT = dbo.fn_GetCompletionDelaySeconds(@Amount);
SET @ReadyToCompleteAt = DATEADD(SECOND, @DelaySeconds, GETDATE());

INSERT INTO dbo.Transactions
    (TransactionTypeID, FromAccountID, ToAccountID, EmployeeID, Amount,
     TransactionStatus, ReadyToCompleteAt, CompletedAt, Description)
VALUES
    (@TransactionTypeID, NULL, @AccountID, @EffectiveEmployeeID, @Amount,
     'Pending', @ReadyToCompleteAt, NULL, @Description);

```

6.2 Check available funds and queue a withdrawal

Database object	dbo.sp_Transaction_Withdrawal
Source file	StoredProcedures/TransactionProcedures/ Withdraw_SP.sql
Source lines	67-96

Function. Locks the source account, subtracts pending outgoing holds from posted balance, enforces the product minimum balance, and inserts a Pending Withdrawal.

```

SELECT @CurrentBalance = Balance, @AccountStatus = AccountStatus, @AccountTypeID = AccountTypeID
FROM dbo.Account WITH (UPDLOCK, HOLDLOCK)
WHERE AccountID = @AccountID;

IF @AccountStatus <> 'Active'
BEGIN RAISERROR('Withdrawals are only allowed on an Active account.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;

DECLARE @MinBalance DECIMAL(18,2), @PendingOutgoingAmount DECIMAL(18,2), @AvailableBalance DECIMAL(18,2);
SELECT @MinBalance = MinBalance FROM dbo.AccountType WHERE AccountTypeID = @AccountTypeID;
SELECT @PendingOutgoingAmount = PendingOutgoingAmount FROM dbo.vw_AccountPendingOutgoing WHERE AccountID = @AccountID;
SET @PendingOutgoingAmount = ISNULL(@PendingOutgoingAmount, 0);
SET @AvailableBalance = @CurrentBalance - @PendingOutgoingAmount;

IF (@AvailableBalance - @Amount) < @MinBalance
BEGIN RAISERROR('Insufficient available funds: withdrawal would breach minimum balance.', 16, 1); ROLLBACK TRANSACTION; RETURN;
END;

DECLARE @TransactionTypeID INT;
SELECT @TransactionTypeID = TransactionTypeID FROM dbo.TransactionType WHERE TypeName = 'Withdrawal';
IF @TransactionTypeID IS NULL
BEGIN RAISERROR('Withdrawal transaction type is not configured.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;

DECLARE @DelaySeconds INT = dbo.fn_GetCompletionDelaySeconds(@Amount);
SET @ReadyToCompleteAt = DATEADD(SECOND, @DelaySeconds, GETDATE());

INSERT INTO dbo.Transactions
    (TransactionTypeID, FromAccountID, ToAccountID, EmployeeID, Amount,
     TransactionStatus, ReadyToCompleteAt, CompletedAt, Description)
VALUES
    (@TransactionTypeID, @AccountID, NULL, @EffectiveEmployeeID, @Amount,
     'Pending', @ReadyToCompleteAt, NULL, @Description);

```

6.3 Lock both accounts and queue a transfer

Database object	dbo.sp_Transaction_Transfer
------------------------	-----------------------------

Source file	StoredProcedures/TransactionProcedures/Transfer_SP.sql
Source lines	72-111

Function. Locks the source and destination in a fixed order, validates both states and source available balance, and inserts a Pending Transfer.

```

SELECT @Dummy = AccountID FROM dbo.Account WITH (UPDLOCK, HOLDLOCK)
WHERE AccountID IN (@LowID, @HighID)
ORDER BY AccountID;

DECLARE @FromBalance DECIMAL(18,2), @FromStatus NVARCHAR(20), @FromTypeID INT, @ToStatus NVARCHAR(20);
SELECT @FromBalance = Balance, @FromStatus = AccountStatus, @FromTypeID = AccountTypeID FROM dbo.Account WHERE AccountID =
@FromAccountID;
SELECT @ToStatus = AccountStatus FROM dbo.Account WHERE AccountID = @ToAccountID;

IF @FromBalance IS NULL
BEGIN RAISERROR('Source account does not exist.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;
IF @ToStatus IS NULL
BEGIN RAISERROR('Destination account does not exist.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;
IF @FromStatus <> 'Active'
BEGIN RAISERROR('Source account must be Active to send a transfer.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;
IF @ToStatus NOT IN ('Active', 'Dormant')
BEGIN RAISERROR('Destination account cannot be Closed or Frozen.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;

DECLARE @MinBalance DECIMAL(18,2), @PendingOutgoingAmount DECIMAL(18,2), @AvailableBalance DECIMAL(18,2);
SELECT @MinBalance = MinBalance FROM dbo.AccountType WHERE AccountTypeID = @FromTypeID;
SELECT @PendingOutgoingAmount = PendingOutgoingAmount FROM dbo.vw_AccountPendingOutgoing WHERE AccountID =
@FromAccountID;
SET @PendingOutgoingAmount = ISNULL(@PendingOutgoingAmount, 0);
SET @AvailableBalance = @FromBalance - @PendingOutgoingAmount;

IF (@AvailableBalance - @Amount) < @MinBalance
BEGIN RAISERROR('Insufficient available funds: transfer would breach minimum balance.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;

DECLARE @TransactionTypeID INT;
SELECT @TransactionTypeID = TransactionTypeID FROM dbo.TransactionType WHERE TypeName = 'Transfer';
IF @TransactionTypeID IS NULL
BEGIN RAISERROR('Transfer transaction type is not configured.', 16, 1); ROLLBACK TRANSACTION; RETURN; END;

DECLARE @DelaySeconds INT = dbo.fn_GetCompletionDelaySeconds(@Amount);
SET @ReadyToCompleteAt = DATEADD(SECOND, @DelaySeconds, GETDATE());

INSERT INTO dbo.Transactions
(TransactionTypeID, FromAccountID, ToAccountID, EmployeeID, Amount,
TransactionStatus, ReadyToCompleteAt, CompletedAt, Description)
VALUES
(@TransactionTypeID, @FromAccountID, @ToAccountID, @EffectiveEmployeeID, @Amount,
'Pending', @ReadyToCompleteAt, NULL, @Description);

```

6.4 Enforce transaction readiness before finalization

Database object	dbo.sp_Transaction_Finalize
Source file	StoredProcedures/TransactionProcedures/TransactionFinalizer.sql
Source lines	123-145

Function. Returns the existing result for a non-Pending transaction and keeps a transaction Pending until ReadyToCompleteAt has been reached.

```

IF @TransactionStatus <> 'Pending'
BEGIN
-- Not an error: calling Finalize twice on an already
-- Completed/Failed/Cancelled transaction is harmless,
-- it just reports the existing outcome.

```



```

SET @ResultStatus = @TransactionStatus;
SET @ResultMessage = CONCAT('Transaction is already ', @TransactionStatus, '.');
ROLLBACK TRANSACTION;
RETURN;
END;

```

```

-----
-- Enforce the time-to-complete hold
-----

```

```

IF GETDATE() < @ReadyToCompleteAt
BEGIN
    SET @ResultStatus = 'StillPending';
    SET @ResultMessage = CONCAT(
        'Not ready yet. Earliest completion time: ',
        CONVERT(VARCHAR(30), @ReadyToCompleteAt, 121)
    );
    ROLLBACK TRANSACTION;
    RETURN;

```

6.5 Revalidate the source account under lock

Database object	dbo.sp_Transaction_Finalize
Source file	StoredProcedures/TransactionProcedures/ TransactionFinalizer.sql
Source lines	168-224

Function. Locks affected accounts in a consistent order and marks the transaction Failed when the source is no longer Active or no longer satisfies its minimum balance.

```

SELECT @Dummy = AccountID FROM dbo.Account WITH (UPDLOCK, HOLDLOCK)
WHERE AccountID IN (@LowID, @HighID)
ORDER BY AccountID;

-----
-- Re-validate the SOURCE side, if any (Withdrawal/Transfer)
-----

IF @FromAccountID IS NOT NULL
BEGIN
    DECLARE
        @FromBalance DECIMAL(18,2),
        @FromStatus NVARCHAR(20),
        @FromTypeID INT,
        @FromMinBalance DECIMAL(18,2);

    SELECT
        @FromBalance = Balance,
        @FromStatus = AccountStatus,
        @FromTypeID = AccountTypeID
    FROM dbo.Account
    WHERE AccountID = @FromAccountID;

    IF @FromStatus IS NULL OR @FromStatus <> 'Active'
    BEGIN
        UPDATE dbo.Transactions
        SET TransactionStatus = 'Failed'
        WHERE TransactionID = @TransactionID;

        SET @ResultStatus = 'Failed';
        SET @ResultMessage = 'Source account is no longer Active; transaction marked Failed.';

        INSERT INTO dbo.AuditLog (UserID, ActionType, TableName, RecordID, Details)
        VALUES (@UserID, 'TransactionFinalizeFailed', 'Transactions', @TransactionID, @ResultMessage);

        COMMIT TRANSACTION;
        RETURN;
    END;

    SELECT @FromMinBalance = MinBalance
    FROM dbo.AccountType

```

```

WHERE AccountTypeID = @FromTypeID;

IF (@FromBalance - @Amount) < @FromMinBalance
BEGIN
    UPDATE dbo.Transactions
    SET TransactionStatus = 'Failed'
    WHERE TransactionID = @TransactionID;

    SET @ResultStatus = 'Failed';
    SET @ResultMessage = 'Source account no longer has sufficient available balance; transaction marked Failed.';

    INSERT INTO dbo.AuditLog (UserID, ActionType, TableName, RecordID, Details)
    VALUES (@UserID, 'TransactionFinalizeFailed', 'Transactions', @TransactionID, @ResultMessage);

    COMMIT TRANSACTION;
    RETURN;
END;

```

6.6 Post balances, reactivate a dormant destination, and complete the transaction

Database object	dbo.sp_Transaction_Finalize
Source file	StoredProcedures/TransactionProcedures/ TransactionFinalizer.sql
Source lines	258-281

Function. Applies the debit and credit effects, changes a Dormant destination to Active, and marks the transaction Completed.

```

IF @FromAccountID IS NOT NULL
BEGIN
    UPDATE dbo.Account
    SET Balance = Balance - @Amount
    WHERE AccountID = @FromAccountID;
END;

DECLARE @ReactivatedToAccount BIT = 0;

IF @ToAccountID IS NOT NULL
BEGIN
    IF EXISTS (SELECT 1 FROM dbo.Account WHERE AccountID = @ToAccountID AND AccountStatus = 'Dormant')
        SET @ReactivatedToAccount = 1;

    UPDATE dbo.Account
    SET Balance = Balance + @Amount,
        AccountStatus = CASE WHEN AccountStatus = 'Dormant' THEN 'Active' ELSE AccountStatus END
    WHERE AccountID = @ToAccountID;
END;

UPDATE dbo.Transactions
SET TransactionStatus = 'Completed',
    CompletedAt = GETDATE()
WHERE TransactionID = @TransactionID;

```

6.7 Cancel a transaction that is still Pending

Database object	dbo.sp_Transaction_Reverse
Source file	StoredProcedures/TransactionProcedures/ Procedure_ReverseTransaction.sql
Source lines	148-159

Function. Changes a Pending transaction to Cancelled before any balance effect has been posted.

```

IF @CurrentStatus = N'Pending'
BEGIN
    UPDATE dbo.Transactions
    SET TransactionStatus = N'Cancelled',
        CompletedAt = NULL,
        Description = CONCAT(
            ISNULL(Description, N''),
            N' [Cancelled before completion: ',
            ISNULL(@ReasonDescription, N'no reason given'),
            N']'
        )
    WHERE TransactionID = @TransactionID;

```

6.8 Reverse the balance effects of a completed transaction

Database object	dbo.sp_Transaction_Reverse
Source file	StoredProcedures/TransactionProcedures/ Procedure_ReverseTransaction.sql
Source lines	215-267

Function. Checks the credited account minimum balance, restores the debit side, removes the credit side, and changes the original transaction to Cancelled.

```

SELECT
    @ToBalance = Balance,
    @ToTypeID = AccountTypeID
FROM dbo.Account WITH (UPDLOCK, HOLDLOCK)
WHERE AccountID = @ToAccountID;

IF @ToBalance IS NULL
BEGIN
    RAISERROR('Destination account does not exist.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

SELECT @ToMinBalance = MinBalance
FROM dbo.AccountType
WHERE AccountTypeID = @ToTypeID;

-- Reversing a Deposit/Transfer takes money away from the
-- credited account, so minimum balance is checked again.
IF (@ToBalance - @Amount) < @ToMinBalance
BEGIN
    RAISERROR('Cannot reverse: destination account would fall below its minimum balance.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

-- Undo debit side: Withdrawal/Transfer source gets money back.
IF @FromAccountID IS NOT NULL
BEGIN
    UPDATE dbo.Account
    SET Balance = Balance + @Amount
    WHERE AccountID = @FromAccountID;
END;

-- Undo credit side: Deposit/Transfer destination loses money.
IF @ToAccountID IS NOT NULL
BEGIN
    UPDATE dbo.Account
    SET Balance = Balance - @Amount
    WHERE AccountID = @ToAccountID;
END;

UPDATE dbo.Transactions
SET TransactionStatus = N'Cancelled',
    CompletedAt = NULL,

```

```

Description = CONCAT(
    ISNULL(Description, N''),
    N' [Reversed: ',
    ISNULL(@ReasonDescription, N'no reason given'),
    N']'
)
WHERE TransactionID = @TransactionID;

```

7. Loans and installments

7.1 Create a loan and generate its installment schedule

Database object	dbo.sp_Loan_Create
Source file	StoredProcedures/LoanProcedures/ Procedure_Loan_Create.sql
Source lines	31-35

Function. Inserts the Active loan and generates the complete installment schedule through a recursive sequence.

```

INSERT INTO dbo.Loan(CustomerID,BranchID,LoanAmount,InterestRate,StartDate,EndDate,LoanStatus)
VALUES(@CustomerID,@BranchID,@LoanAmount,@InterestRate,@StartDate,DATEADD(MONTH,@NumberOfInstallments,@StartDate),'Active');
SET @LoanID=CONVERT(INT,SCOPE_IDENTITY());
;WITH Seq AS(SELECT 1 AS InstallmentNumber UNION ALL SELECT InstallmentNumber+1 FROM Seq WHERE
InstallmentNumber<@NumberOfInstallments)
INSERT INTO dbo.Installment(LoanID,DueDate,Amount,PaidDate,InstallmentStatus,PaymentTransactionID)
SELECT @LoanID,DATEADD(MONTH,InstallmentNumber,@StartDate),CASE WHEN InstallmentNumber=@NumberOfInstallments THEN
@LastInstallmentAmt ELSE @BaseInstallmentAmt END,NULL,'Pending',NULL FROM Seq OPTION(MAXRECURSION 1000);

```

7.2 Return loan totals and calculated installment display status

Database object	dbo.sp_Loan_GetStatus
Source file	StoredProcedures/LoanProcedures/ Procedure_Loan_GetStatus.sql
Source lines	26-29

Function. Returns an aggregate loan summary and a second result set in which DisplayStatus reflects payment processing and the current date while StoredStatus remains the persisted value.

```

SELECT l.LoanID,l.CustomerID,c.FirstName,c.LastName,l.BranchID,l.LoanAmount,l.InterestRate,l.LoanAmount*(1+l.InterestRate/100.0) AS
TotalToRepay,l.StartDate,l.EndDate,l.LoanStatus,COUNT(i.InstallmentID) AS TotalInstallments,SUM(CASE WHEN i.InstallmentStatus='Paid'
THEN 1 ELSE 0 END) AS PaidInstallments,SUM(CASE WHEN i.InstallmentStatus='Paid' THEN i.Amount ELSE 0 END) AS AmountPaid,SUM(CASE
WHEN i.InstallmentStatus<>'Paid' THEN i.Amount ELSE 0 END) AS AmountRemaining,SUM(CASE WHEN i.InstallmentStatus<>'Paid' AND
i.DueDate<CAST(GETDATE() AS DATE) THEN 1 ELSE 0 END) AS OverdueInstallments
FROM dbo.Loan l JOIN dbo.Customer c ON l.CustomerID=c.CustomerID LEFT JOIN dbo.Installment i ON i.LoanID=l.LoanID WHERE
l.LoanID=@LoanID GROUP BY
l.LoanID,l.CustomerID,c.FirstName,c.LastName,l.BranchID,l.LoanAmount,l.InterestRate,l.StartDate,l.EndDate,l.LoanStatus;
SELECT i.InstallmentID,i.DueDate,i.Amount,i.PaidDate,i.InstallmentStatus AS StoredStatus,CASE WHEN i.InstallmentStatus IN ('Paid','Defaulted')
THEN i.InstallmentStatus WHEN tr.TransactionStatus='Pending' THEN 'PaymentPending' WHEN i.DueDate<CAST(GETDATE() AS DATE) THEN
'Late' ELSE 'Pending' END AS DisplayStatus,i.PaymentTransactionID,tr.TransactionStatus AS PaymentTransactionStatus,tr.ReadyToCompleteAt
AS PaymentReadyToCompleteAt
FROM dbo.Installment i LEFT JOIN dbo.Transactions tr ON tr.TransactionID=i.PaymentTransactionID WHERE i.LoanID=@LoanID ORDER BY
i.DueDate;

```

7.3 Associate an installment with its payment transaction

Database object	dbo.sp_Loan_PayInstallment
Source file	StoredProcedures/LoanProcedures/ Procedure_Loan_PayInstallment.sql
Source lines	24-37

Function. Checks installment and borrower-account relationships, prevents duplicate pending payments, and stores the withdrawal transaction used to pay the installment.

```

SELECT @LoanID=i.LoanID,@Amount=i.Amount,@InstallmentStatus=i.InstallmentStatus,@ExistingPaymentTxID=i.PaymentTransactionID
FROM dbo.Installment i WHERE i.InstallmentID=@InstallmentID;
SELECT @CallerCustomerID=CustomerID FROM dbo.Users WHERE UserID=@UserID;
IF @LoanID IS NULL BEGIN RAISERROR('Installment does not exist.',16,1); ROLLBACK TRANSACTION; RETURN; END;
IF @InstallmentStatus='Paid' BEGIN RAISERROR('This installment has already been paid.',16,1); ROLLBACK TRANSACTION; RETURN; END;
IF @InstallmentStatus='Defaulted' BEGIN RAISERROR('This installment has been marked Defaulted and can no longer be paid directly.',16,1);
ROLLBACK TRANSACTION; RETURN; END;
IF @ExistingPaymentTxID IS NOT NULL AND EXISTS(SELECT 1 FROM dbo.Transactions WHERE TransactionID=@ExistingPaymentTxID AND
TransactionStatus='Pending') BEGIN RAISERROR('A payment for this installment is already pending completion.',16,1); ROLLBACK
TRANSACTION; RETURN; END;
SELECT @LoanCustomerID=CustomerID,@LoanStatus=LoanStatus FROM dbo.Loan WHERE LoanID=@LoanID;
IF @LoanStatus<>'Active' BEGIN RAISERROR('This loan is not Active; no further payments are expected.',16,1); ROLLBACK TRANSACTION;
RETURN; END;
IF @LoanCustomerID<>@CallerCustomerID BEGIN RAISERROR('Only the borrower can pay this loan installment.',16,1); ROLLBACK
TRANSACTION; RETURN; END;
IF NOT EXISTS(SELECT 1 FROM dbo.Account WHERE AccountID=@FromAccountID AND CustomerID=@LoanCustomerID) BEGIN
RAISERROR('The paying account does not belong to the customer who owns this loan.',16,1); ROLLBACK TRANSACTION; RETURN; END;
EXEC dbo.sp_Transaction-Withdrawal
@AccountID=@FromAccountID,@Amount=@Amount,@EmployeeID=@EmployeeID,@Description=N'Loan installment
payment',@TransactionID=@TransactionID OUTPUT,@ReadyToCompleteAt=@ReadyToCompleteAt OUTPUT,@UserID=@UserID;
IF @@TRANCOUNT=0 BEGIN RAISERROR('Installment payment could not be initiated: the withdrawal was rejected.',16,1); RETURN; END;
UPDATE dbo.Installment SET PaymentTransactionID=@TransactionID WHERE InstallmentID=@InstallmentID;
INSERT INTO dbo.AuditLog(UserID,ActionType,TableName,RecordID,Details)
VALUES(@UserID,'InstallmentPaymentInitiated','Installment',@InstallmentID,CONCAT('Payment of ',@Amount,' queued via TransactionID
',@TransactionID,'; ready at ',CONVERT(VARCHAR(30),@ReadyToCompleteAt,121)));

```

7.4 Classify overdue installments and update their stored states

Database object	dbo.sp_Loan_ProcessOverdueInstallments
Source file	StoredProcedures/LoanProcedures/ Procedure_Loan_ProcessOverdueInstallments. sql
Source lines	101-175

Function. Builds separate candidate sets for Defaulted, Late, and restored Pending installments, then applies the corresponding state updates.

```

SELECT
    I.InstallmentID,
    I.LoanID,
    L.CustomerID,
    L.BranchID,
    I.DueDate,
    I.Amount,
    I.InstallmentStatus AS OldStatus
INTO #ToDefault
FROM dbo.Installment AS I WITH (UPDLOCK, HOLDLOCK)
INNER JOIN dbo.Loan AS L WITH (UPDLOCK, HOLDLOCK)
    ON L.LoanID = I.LoanID
LEFT JOIN dbo.Transactions AS T
    ON T.TransactionID = I.PaymentTransactionID
WHERE I.InstallmentStatus NOT IN (N'Paid', N'Defaulted')
AND I.DueDate < @DefaultCutoff
AND (T.TransactionID IS NULL OR T.TransactionStatus <> N'Pending')

```

```

AND (@EffectiveBranchID IS NULL OR L.BranchID = @EffectiveBranchID);

SELECT
    I.InstallmentID,
    I.LoanID,
    L.CustomerID,
    L.BranchID,
    I.DueDate,
    I.Amount,
    I.InstallmentStatus AS OldStatus
INTO #ToLate
FROM dbo.Installment AS I WITH (UPDLOCK, HOLDLOCK)
INNER JOIN dbo.Loan AS L WITH (UPDLOCK, HOLDLOCK)
    ON L.LoanID = I.LoanID
LEFT JOIN dbo.Transactions AS T
    ON T.TransactionID = I.PaymentTransactionID
WHERE I.InstallmentStatus = N'Pending'
    AND I.DueDate < @Today
    AND I.DueDate >= @DefaultCutoff
    AND (T.TransactionID IS NULL OR T.TransactionStatus <> N'Pending')
    AND (@EffectiveBranchID IS NULL OR L.BranchID = @EffectiveBranchID);

SELECT
    I.InstallmentID,
    I.LoanID,
    L.CustomerID,
    L.BranchID,
    I.DueDate,
    I.Amount,
    I.InstallmentStatus AS OldStatus
INTO #ToPending
FROM dbo.Installment AS I WITH (UPDLOCK, HOLDLOCK)
INNER JOIN dbo.Loan AS L WITH (UPDLOCK, HOLDLOCK)
    ON L.LoanID = I.LoanID
LEFT JOIN dbo.Transactions AS T
    ON T.TransactionID = I.PaymentTransactionID
WHERE I.InstallmentStatus = N'Late'
    AND I.DueDate >= @Today
    AND (T.TransactionID IS NULL OR T.TransactionStatus <> N'Pending')
    AND (@EffectiveBranchID IS NULL OR L.BranchID = @EffectiveBranchID);

UPDATE I
SET I.InstallmentStatus = N'Defaulted'
FROM dbo.Installment AS I
INNER JOIN #ToDefault AS D
    ON D.InstallmentID = I.InstallmentID;

UPDATE I
SET I.InstallmentStatus = N'Late'
FROM dbo.Installment AS I
INNER JOIN #ToLate AS D
    ON D.InstallmentID = I.InstallmentID;

UPDATE I
SET I.InstallmentStatus = N'Pending'
FROM dbo.Installment AS I
INNER JOIN #ToPending AS D
    ON D.InstallmentID = I.InstallmentID;

```

7.5 Default loans, mark fully paid loans, and freeze borrower accounts

Database object	dbo.sp_Loan_ProcessOverdueInstallments
Source file	StoredProcedures/LoanProcedures/ Procedure_Loan_ProcessOverdueInstallments. sql
Source lines	177-252

Function. Changes Active loans to Defaulted when any installment is Defaulted, changes fully paid loans to Paid, and freezes every Active or Dormant account belonging to a newly defaulted borrower.

```

SELECT DISTINCT
    L.LoanID,
    L.CustomerID,
    L.BranchID,
    L.LoanStatus AS OldStatus
INTO #LoansToDefault
FROM dbo.Loan AS L WITH (UPDLOCK, HOLDLOCK)
WHERE L.LoanStatus = N'Active'
AND (@EffectiveBranchID IS NULL OR L.BranchID = @EffectiveBranchID)
AND EXISTS
(
    SELECT 1
    FROM dbo.Installment AS I
    WHERE I.LoanID = L.LoanID
    AND I.InstallmentStatus = N'Defaulted'
);

UPDATE L
SET L.LoanStatus = N'Defaulted'
FROM dbo.Loan AS L
INNER JOIN #LoansToDefault AS D
    ON D.LoanID = L.LoanID;

SELECT
    L.LoanID,
    L.CustomerID,
    L.BranchID,
    L.LoanStatus AS OldStatus
INTO #LoansToPaid
FROM dbo.Loan AS L WITH (UPDLOCK, HOLDLOCK)
WHERE L.LoanStatus = N'Active'
AND (@EffectiveBranchID IS NULL OR L.BranchID = @EffectiveBranchID)
AND EXISTS
(
    SELECT 1
    FROM dbo.Installment AS I
    WHERE I.LoanID = L.LoanID
)
AND NOT EXISTS
(
    SELECT 1
    FROM dbo.Installment AS I
    WHERE I.LoanID = L.LoanID
    AND I.InstallmentStatus <> N'Paid'
);

UPDATE L
SET L.LoanStatus = N'Paid'
FROM dbo.Loan AS L
INNER JOIN #LoansToPaid AS P
    ON P.LoanID = L.LoanID;

SELECT
    A.AccountID,
    A.CustomerID,
    A.BranchID,
    A.AccountStatus AS OldStatus,
    MIN(D.LoanID) AS LoanID
INTO #AccountsToFreeze
FROM dbo.Account AS A WITH (UPDLOCK, HOLDLOCK)
INNER JOIN #LoansToDefault AS D
    ON D.CustomerID = A.CustomerID
WHERE A.AccountStatus IN (N'Active', N'Dormant')
GROUP BY
    A.AccountID,
    A.CustomerID,
    A.BranchID,
    A.AccountStatus;

UPDATE A
SET

```

```

A.FrozenPreviousStatus = A.AccountStatus,
A.AccountStatus = N'Frozen'
FROM dbo.Account AS A
INNER JOIN #AccountsToFreeze AS F
ON F.AccountID = A.AccountID;

```

8. Employees, managers, and transfers

8.1 Suspend an ordinary employee

Database object	dbo.sp_Employee_Suspend
Source file	StoredProcedures/Employees/ Procedure_Employee_Suspend.sql
Source lines	155-178

Function. Changes EmpStatus to OnLeave and records the user who performed the suspension.

```

UPDATE dbo.Employee
SET EmpStatus = N'OnLeave'
WHERE EmployeeID = @EmployeeID;

INSERT INTO dbo.AuditLog
(
    UserID,
    ActionType,
    TableName,
    RecordID,
    Details
)
VALUES
(
    @ManagerUserID,
    N'EmployeeSuspended',
    N'Employee',
    @EmployeeID,
    CONCAT(
        N'Employee suspended in BranchID=', @TargetBranchID,
        N'; SuspendedBy=', CASE WHEN @IsHighAdmin = 1 THEN N'HighAdmin' ELSE N'BranchManager' END,
        N'; Reason=', ISNULL(NULLIF(LTRIM(RTRIM(@Reason)), N''), N'not provided')
    )
);

```

8.2 Verify that the same user reverses the latest employee suspension

Database object	dbo.sp_Employee_Unsuspend
Source file	StoredProcedures/Employees/ Procedure_Employee_Unsuspend.sql
Source lines	91-112

Function. Reads the latest EmployeeSuspended audit record and rejects reactivation when it is requested by a different user.

```

SELECT TOP (1)
    @LatestSuspensionAuditID = A.AuditID,
    @LatestSuspendedByUserID = A.UserID
FROM dbo.AuditLog AS A WITH (UPDLOCK, HOLDLOCK)
WHERE A.TableName = N'Employee'
AND A.RecordID = @EmployeeID
AND A.ActionType = N'EmployeeSuspended'

```



```

ORDER BY A.ActionDate DESC, A.AuditID DESC;

IF @LatestSuspensionAuditID IS NULL
BEGIN
    RAISERROR('No suspension record exists for this employee.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

IF @LatestSuspendedByUserID <> @ManagerUserID
BEGIN
    RAISERROR('Only the manager or HighAdmin user who performed the latest suspension can reverse it.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

```

8.3 Reactivate an ordinary employee

Database object	dbo.sp_Employee_Unsuspend
Source file	StoredProcedures/Employees/ Procedure_Employee_Unsuspend.sql
Source lines	161-184

Function. Returns the employee to Active and records the suspension audit entry that was reversed.

```

UPDATE dbo.Employee
SET EmpStatus = N'Active'
WHERE EmployeeID = @EmployeeID;

INSERT INTO dbo.AuditLog
(
    UserID,
    ActionType,
    TableName,
    RecordID,
    Details
)
VALUES
(
    @ManagerUserID,
    N'EmployeeUnsuspended',
    N'Employee',
    @EmployeeID,
    CONCAT(
        N'Employee restored to Active in BranchID=', @TargetBranchID,
        N'; ReversedSuspensionAuditID=', @LatestSuspensionAuditID,
        N'; Reason=', ISNULL(NULLIF(LTRIM(RTRIM(@Reason)), N'), N'not provided')
    )
);

```

8.4 Reactivate a manager and restore the Admin role

Database object	dbo.sp_HighAdmin_UnsuspendManager
Source file	StoredProcedures/Employees/ Procedure_HighAdmin_UnsuspendManager.sql
Source lines	81-150

Function. Requires the same HighAdmin who performed the latest suspension, prevents duplicate active Branch Managers, restores Active status, and inserts the Admin role when missing.

```

SELECT TOP (1)
    @LatestSuspensionAuditID = A.AuditID,
    @LatestSuspendedByUserID = A.UserID
FROM dbo.AuditLog AS A WITH (UPDLCK, HOLDLOCK)
WHERE A.TableName = N'Employee'
    AND A.RecordID = @ManagerEmployeeID
    AND A.ActionType = N'ManagerSuspended'
ORDER BY A.ActionDate DESC, A.AuditID DESC;

IF @LatestSuspensionAuditID IS NULL
BEGIN
    RAISERROR('No manager suspension record exists for this employee.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

IF @LatestSuspendedByUserID <> @HighAdminUserID
BEGIN
    RAISERROR('Only the HighAdmin user who performed the latest suspension can reverse it.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

IF @JobTitle = N'Branch Manager'
    AND EXISTS
    (
        SELECT 1
        FROM dbo.EMPB AS EB
        INNER JOIN dbo.Employee AS E
            ON E.EmployeeID = EB.EmployeeID
        WHERE EB.BranchID = @BranchID
            AND EB.WorkingStatus = N'Working'
            AND EB.EndDate IS NULL
            AND E.EmpStatus = N'Active'
            AND E.JobTitle = N'Branch Manager'
            AND E.EmployeeID <> @ManagerEmployeeID
    )
BEGIN
    RAISERROR('This Branch Manager cannot be reactivated because the branch already has another active Branch Manager.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

UPDATE dbo.Employee
SET EmpStatus = N'Active'
WHERE EmployeeID = @ManagerEmployeeID;

SELECT @AdminRoleID = R.RoleID
FROM dbo.Roles AS R
WHERE R.RoleName = N'Admin';

IF @AdminRoleID IS NULL
BEGIN
    RAISERROR('Admin role is missing from the role catalogue.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

INSERT INTO dbo.UserRoles (UserID, RoleID)
SELECT U.UserID, @AdminRoleID
FROM dbo.Users AS U
WHERE U.EmployeeID = @ManagerEmployeeID
    AND U.IsActive = 1
    AND NOT EXISTS
    (
        SELECT 1
        FROM dbo.UserRoles AS UR
        WHERE UR.UserID = U.UserID
            AND UR.RoleID = @AdminRoleID
    );

```

8.5 Complete an employee branch transfer

Database object	dbo.sp_EmployeeTransfer_ApproveDestinationManager
Source file	StoredProcedures/Employees/Procedure_EmployeeTransfer_ApproveDestinationManager.sql
Source lines	174-234

Function. Locks the current assignment, changes it to Transferred, inserts the new Working assignment, and marks the transfer request Completed.

```

SELECT TOP (1)
    @CurrentEMPBID = EB.EMPBID
FROM dbo.EMPB AS EB WITH (UPDLOCK, HOLDLOCK)
WHERE EB.EmployeeID = @EmployeeID
    AND EB.BranchID = @FromBranchID
    AND EB.WorkingStatus = 'Working'
    AND EB.EndDate IS NULL;

IF @CurrentEMPBID IS NULL
BEGIN
    RAISERROR('Employee no longer has the expected current branch assignment.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

IF NOT EXISTS
(
    SELECT 1
    FROM dbo.Employee AS E WITH (UPDLOCK, HOLDLOCK)
    WHERE E.EmployeeID = @EmployeeID
        AND E.EmpStatus = 'Active'
)
BEGIN
    RAISERROR('Employee is no longer active.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;

-- End previous branch assignment first.
UPDATE dbo.EMPB
SET EndDate = @EffectiveDate,
    WorkingStatus = 'Transferred'
WHERE EMPBID = @CurrentEMPBID;

-- Start destination branch assignment.
INSERT INTO dbo.EMPB
(
    EmployeeID,
    BranchID,
    StartDate,
    EndDate,
    WorkingStatus
)
VALUES
(
    @EmployeeID,
    @ToBranchID,
    @EffectiveDate,
    NULL,
    'Working'
);

UPDATE dbo.EmployeeTransferRequest
SET DestinationManagerUserID = @ManagerUserID,
    DestinationManagerDecision = 'Approved',
    DestinationManagerDecisionDate = GETDATE(),
    DestinationManagerNote = @DecisionNote,
    TransferStatus = 'Completed',
    EffectiveDate = @EffectiveDate,

```

```
CompletedAt = GETDATE()
WHERE TransferRequestID = @TransferRequestID;
```

9. Integrity triggers

9.1 Prevent closing an account with a pending transaction

Database object	dbo.TR_Account_PreventCloseWithPending
Source file	Triggers/AccountTriggers/ Trigger_Account_PreventCloseWithPending.sql
Source lines	27-43

Function. Uses inserted and deleted to detect a transition to Closed and rejects it when a Pending transaction references the account.

```
IF EXISTS (
    SELECT 1
    FROM inserted i
    INNER JOIN deleted d ON i.AccountID = d.AccountID
    WHERE i.AccountStatus = 'Closed'
    AND d.AccountStatus <> 'Closed'
    AND EXISTS (
        SELECT 1 FROM dbo.Transactions tr
        WHERE tr.TransactionStatus = 'Pending'
        AND (tr.FromAccountID = i.AccountID OR tr.ToAccountID = i.AccountID)
    )
)
BEGIN
    RAISERROR('Cannot close an account that has a Pending transaction in flight.', 16, 1);
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    RETURN;
END;
```

9.2 Synchronize branch balances and append ledger rows

Database object	dbo.TR_Account_SyncBranchLedger
Source file	Triggers/AccountTriggers/ Trigger_Account_SyncBranchLedger.sql
Source lines	49-95

Function. Calculates account-level balance deltas, aggregates them by branch, updates Branch.Balance, and inserts BranchLedger evidence.

```
IF NOT EXISTS (
    SELECT 1
    FROM inserted i
    LEFT JOIN deleted d ON i.AccountID = d.AccountID
    WHERE i.Balance <> ISNULL(d.Balance, 0)
)
RETURN;

-----
-- Per-account deltas for this batch.
-----

IF OBJECT_ID('tempdb..#AcctDelta') IS NOT NULL DROP TABLE #AcctDelta;

SELECT
    i.AccountID,
```

```

i.BranchID,
i.Balance - ISNULL(d.Balance, 0) AS DeltaAmount
INTO #AcctDelta
FROM inserted i
LEFT JOIN deleted d ON i.AccountID = d.AccountID
WHERE i.Balance <> ISNULL(d.Balance, 0);

-----
-- Apply the net delta per branch to Branch.Balance.
-----
UPDATE b
SET b.Balance = b.Balance + agg.NetDelta
FROM dbo.Branch b
INNER JOIN (
    SELECT BranchID, SUM(DeltaAmount) AS NetDelta
    FROM #AcctDelta
    GROUP BY BranchID
) agg ON agg.BranchID = b.BranchID;

-----
-- One BranchLedger row per account-level delta, stamped
-- with that branch's balance after this batch.
-----
INSERT INTO dbo.BranchLedger (BranchID, TransactionID, DeltaAmount, BalanceAfter, Description)
SELECT
    ad.BranchID,
    NULL,
    ad.DeltaAmount,
    b.Balance,
    CONCAT('Account balance change on AccountID ', ad.AccountID, ' of ', ad.DeltaAmount)
FROM #AcctDelta ad
INNER JOIN dbo.Branch b ON b.BranchID = ad.BranchID;

```

9.3 Prevent customer deactivation while funds are in flight

Database object	dbo.TR_Customer_PreventDeactivateWithPending
Source file	Triggers/CustomerTriggers/ Trigger_Customer_PreventDeactivateWithPending.sql
Source lines	25-44

Function. Rejects an active-to-inactive customer transition when any account owned by the customer participates in a Pending transaction.

```

IF EXISTS (
    SELECT 1
    FROM inserted i
    INNER JOIN deleted d ON i.CustomerID = d.CustomerID
    WHERE i.IsActive = 0
    AND d.IsActive = 1
    AND EXISTS (
        SELECT 1
        FROM dbo.Account a
        INNER JOIN dbo.Transactions tr
            ON (tr.FromAccountID = a.AccountID OR tr.ToAccountID = a.AccountID)
        WHERE a.CustomerID = i.CustomerID
        AND tr.TransactionStatus = 'Pending'
    )
)
BEGIN
    RAISERROR('Cannot deactivate a customer who has an account with a Pending transaction in flight.', 16, 1);
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    RETURN;
END;

```

9.4 Synchronize installment and loan states from payment transactions

Database object	dbo.TR_Transactions_SyncInstallmentStatus
Source file	Triggers/LoanTriggers/ Trigger_SyncInstallmentStatus.sql
Source lines	54-103

Function. Marks installments Paid when linked transactions complete, releases failed payment links, and marks a loan Paid after all installments are Paid.

```

UPDATE i
SET
    i.InstallmentStatus = 'Paid',
    i.PaidDate = CAST(GETDATE() AS DATE)
FROM dbo.Installment i
INNER JOIN inserted ins ON i.PaymentTransactionID = ins.TransactionID
INNER JOIN deleted del ON ins.TransactionID = del.TransactionID
WHERE ins.TransactionStatus = 'Completed'
    AND del.TransactionStatus <> 'Completed';

-----

-- Rows that just became Failed -> release the link so
-- the installment can be attempted again. Status reverts
-- to 'Late' if past due, otherwise stays 'Pending'.

-----

UPDATE i
SET
    i.PaymentTransactionID = NULL,
    i.InstallmentStatus = CASE
        WHEN i.DueDate < CAST(GETDATE() AS DATE) THEN 'Late'
        ELSE 'Pending'
    END
FROM dbo.Installment i
INNER JOIN inserted ins ON i.PaymentTransactionID = ins.TransactionID
INNER JOIN deleted del ON ins.TransactionID = del.TransactionID
WHERE ins.TransactionStatus = 'Failed'
    AND del.TransactionStatus <> 'Failed'
    AND i.InstallmentStatus <> 'Paid'; -- safety: never downgrade an already-Paid row

-----

-- If every installment on a loan is now Paid, mark the
-- loan itself Paid. Scoped to only the loans actually
-- touched by this trigger firing, not a full table scan.

-----

UPDATE l
SET l.LoanStatus = 'Paid'
FROM dbo.Loan l
WHERE l.LoanStatus = 'Active'
    AND EXISTS (
        SELECT 1
        FROM dbo.Installment i2
        INNER JOIN inserted ins2 ON i2.PaymentTransactionID = ins2.TransactionID
        WHERE i2.LoanID = l.LoanID
    )
    AND NOT EXISTS (
        SELECT 1
        FROM dbo.Installment i3
        WHERE i3.LoanID = l.LoanID
        AND i3.InstallmentStatus <> 'Paid'
    );

```

9.5 Enforce account-reference patterns for transaction types

Database object	dbo.TR_Transactions_EnforceTypePattern
Source file	Triggers/TransactionTriggers/ Trigger_Transactions_EnforceTypePattern_Update.sql

Source lines	29-42
---------------------	-------

Function. Rejects rows whose FromAccountID and ToAccountID pattern does not match Deposit, Interest, Withdrawal, or Transfer semantics.

```

IF EXISTS (
    SELECT 1
    FROM inserted i
    INNER JOIN dbo.TransactionType t ON i.TransactionTypeID = t.TransactionTypeID
    WHERE
        (t.TypeName IN ('Deposit', 'Interest') AND (i.FromAccountID IS NOT NULL OR i.ToAccountID IS NULL))
        OR (t.TypeName = 'Withdrawal' AND (i.ToAccountID IS NOT NULL OR i.FromAccountID IS NULL))
        OR (t.TypeName = 'Transfer' AND (i.FromAccountID IS NULL OR i.ToAccountID IS NULL))
)
BEGIN
    RAISERROR('FromAccountID/ToAccountID pattern does not match the transaction type.', 16, 1);
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    RETURN;
END;

```

9.6 Prevent overlapping employee branch assignments

Database object	dbo.trg_EMPB_PreventDateOverlap
Source file	Rules/EMPB_PreventDateOverlap.sql
Source lines	9-29

Function. Compares inserted assignments with existing EMPB rows and rolls back when their date intervals overlap.

```

IF EXISTS (
    SELECT 1
    FROM inserted i
    JOIN dbo.EMPB e
        ON i.EmployeeID = e.EmployeeID
    WHERE
        -- Exclude comparing the exact same record against itself
        -- (Assuming EmployeeID, BranchID, and StartDate form a unique combination for a period)
        NOT (i.BranchID = e.BranchID AND i.StartDate = e.StartDate)

        -- Overlap Logic:
        -- New StartDate must be before the Existing EndDate AND
        -- Existing StartDate must be before the New EndDate.
        -- We use '9999-12-31' to handle NULL EndDates (Currently 'Working' or 'Transferred')
        AND i.StartDate < ISNULL(e.EndDate, '9999-12-31')
        AND e.StartDate < ISNULL(i.EndDate, '9999-12-31')
)
BEGIN
    RAISERROR ('An employee cannot have overlapping work dates across different branches. The previous assignment must end before a new one starts.', 16, 1);
    ROLLBACK TRANSACTION;
END

```

10. Source index

No.	Object	Source file	Lines
1	dbo.sp_User_Login	StoredProcedures/ UserProcedures/ Procedure_User_Login.sql	137-153
2	dbo.sp_User_Login	StoredProcedures/ UserProcedures/ Procedure_User_Login.sql	246-309

No.	Object	Source file	Lines
3	dbo.sp_User_Login	StoredProcedures/ UserProcedures/ Procedure_User_Login.sql	315-348
4	dbo.sp_User_ValidateSession	StoredProcedures/Session/ Procedure_User_ValidateSession .sql	109-167
5	dbo.sp_User_ValidateSession	StoredProcedures/Session/ Procedure_User_ValidateSession .sql	297-385
6	dbo.sp_User_Logout	StoredProcedures/ UserProcedures/ Procedure_User_Logout.sql	48-92
7	dbo.sp_Customer_Search	StoredProcedures/ CustomerProcedures/ Procedure_Customer_Search.sql	136-164
8	dbo.sp_Customer_Update	StoredProcedures/ CustomerProcedures/ Procedure_Customer_Update.sql	114-125
9	dbo.sp_Customer_Delete	StoredProcedures/ CustomerProcedures/ Procedure_Customer_Delete.sql	109-126
10	dbo.sp_Account_Search	StoredProcedures/ AccountProcedures/ Procedure_Account_Search.sql	270-317
11	dbo.sp_Account_Open	StoredProcedures/ AccountProcedures/ Procedure_Account_Open.sql	159-192
12	dbo.sp_Account_Freeze	StoredProcedures/ AccountProcedures/ Procedure_Account_Freeze_Unfr eeze.sql	26-34
13	dbo.sp_Account_Unfreeze	StoredProcedures/ AccountProcedures/ Procedure_Account_Freeze_Unfr eeze.sql	55-63
14	dbo.sp_Account_Close	StoredProcedures/ AccountProcedures/ Procedure_Account_Close.sql	50-55
15	dbo.sp_Account_DormantSweep	StoredProcedures/ AccountProcedures/ Procedure_Account_DormantSw eep.sql	33-76
16	dbo.sp_Account_ApplyMonthlyI nterest	StoredProcedures/ AccountProcedures/ Procedure_Account_ApplyMont hlyInterest.sql	65-123
17	dbo.sp_Transaction_Deposit	StoredProcedures/ TransactionProcedures/ Desposit_SP.sql	67-87
18	dbo.sp_Transaction-Withdrawal	StoredProcedures/ TransactionProcedures/ Withdraw_SP.sql	67-96
19	dbo.sp_Transaction_Transfer	StoredProcedures/ TransactionProcedures/ Transfer_SP.sql	72-111
20	dbo.sp_Transaction_Finalize	StoredProcedures/ TransactionProcedures/ TransactionFinalizer.sql	123-145
21	dbo.sp_Transaction_Finalize	StoredProcedures/ TransactionProcedures/ TransactionFinalizer.sql	168-224

No.	Object	Source file	Lines
22	dbo.sp_Transaction_Finalize	StoredProcedures/ TransactionProcedures/ TransactionFinalizer.sql	258-281
23	dbo.sp_Transaction_Reverse	StoredProcedures/ TransactionProcedures/ Procedure_ReverseTransaction.s ql	148-159
24	dbo.sp_Transaction_Reverse	StoredProcedures/ TransactionProcedures/ Procedure_ReverseTransaction.s ql	215-267
25	dbo.sp_Loan_Create	StoredProcedures/ LoanProcedures/ Procedure_Loan_Create.sql	31-35
26	dbo.sp_Loan_GetStatus	StoredProcedures/ LoanProcedures/ Procedure_Loan_GetStatus.sql	26-29
27	dbo.sp_Loan_PayInstallment	StoredProcedures/ LoanProcedures/ Procedure_Loan_PayInstallment .sql	24-37
28	dbo.sp_Loan_ProcessOverdueIn stallments	StoredProcedures/ LoanProcedures/ Procedure_Loan_ProcessOverdu eInstallments.sql	101-175
29	dbo.sp_Loan_ProcessOverdueIn stallments	StoredProcedures/ LoanProcedures/ Procedure_Loan_ProcessOverdu eInstallments.sql	177-252
30	dbo.sp_Employee_Suspend	StoredProcedures/Employees/ Procedure_Employee_Suspend.s ql	155-178
31	dbo.sp_Employee_Unsuspend	StoredProcedures/Employees/ Procedure_Employee_Unsuspen d.sql	91-112
32	dbo.sp_Employee_Unsuspend	StoredProcedures/Employees/ Procedure_Employee_Unsuspen d.sql	161-184
33	dbo.sp_HighAdmin_Unsuspend Manager	StoredProcedures/Employees/ Procedure_HighAdmin_Unsuspe ndManager.sql	81-150
34	dbo.sp_EmployeeTransfer_Appr oveDestinationManager	StoredProcedures/Employees/ Procedure_EmployeeTransfer_A pproveDestinationManager.sql	174-234
35	dbo.TR_Account_PreventCloseW ithPending	Triggers/AccountTriggers/ Trigger_Account_PreventCloseW ithPending.sql	27-43
36	dbo.TR_Account_SyncBranchLe dger	Triggers/AccountTriggers/ Trigger_Account_SyncBranchLe dger.sql	49-95
37	dbo.TR_Customer_PreventDeacti vateWithPending	Triggers/CustomerTriggers/ Trigger_Customer_PreventDeact ivateWithPending.sql	25-44
38	dbo.TR_Transactions_SyncInstal lmentStatus	Triggers/LoanTriggers/ Trigger_SyncInstallmentStatus.s ql	54-103
39	dbo.TR_Transactions_EnforceTy pePattern	Triggers/TransactionTriggers/ Trigger_Transactions_EnforceTy pePattern_Update.sql	29-42
40	dbo.trg_EMPB_PreventDateOver lap	Rules/ EMPB_PreventDateOverlap.sql	9-29